

NPTEL

NPTEL Course on
"Computer System Optimization for ML"

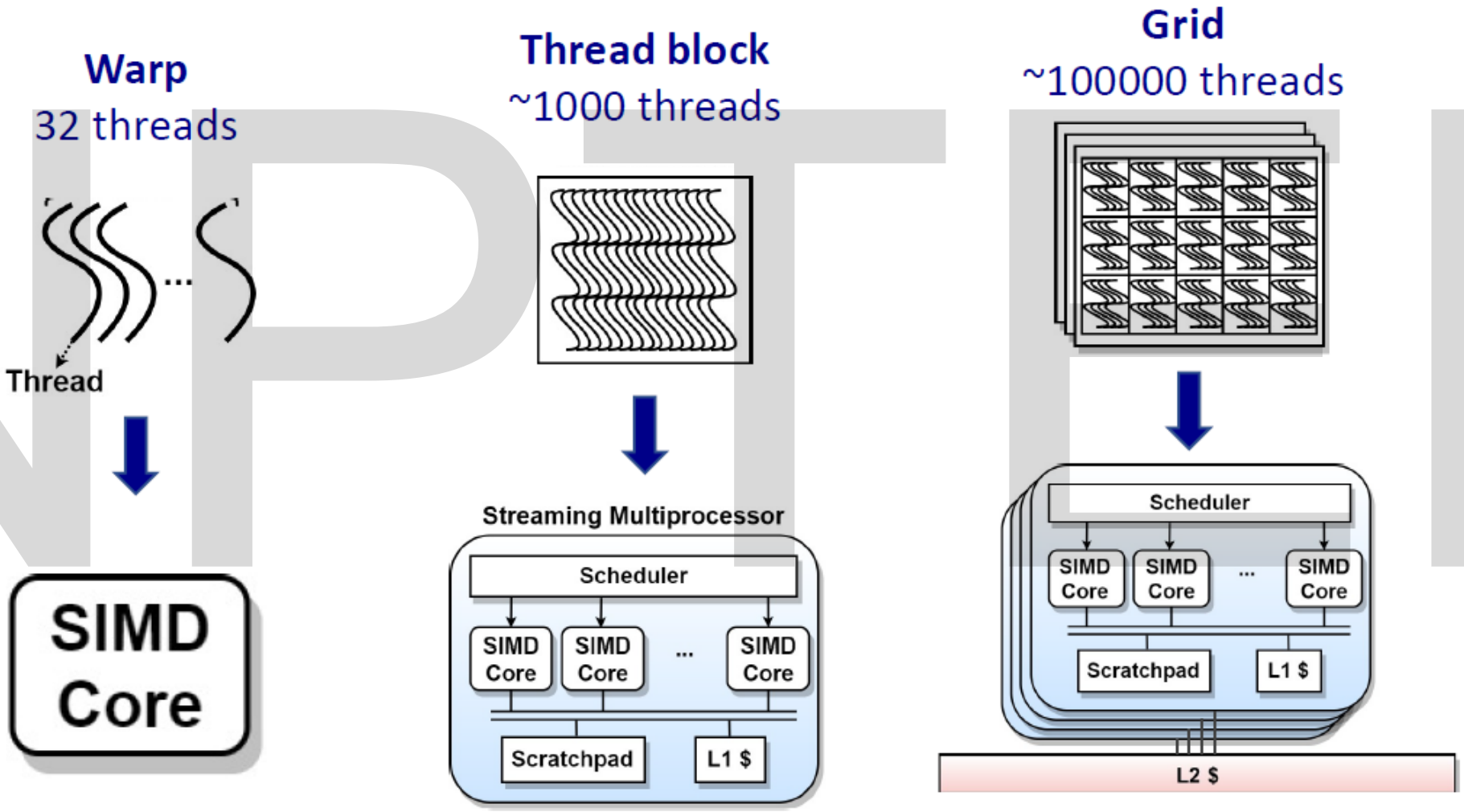
Lecture #59

Agenda

- Recap
- Training Large Models with Limited GPU Memory

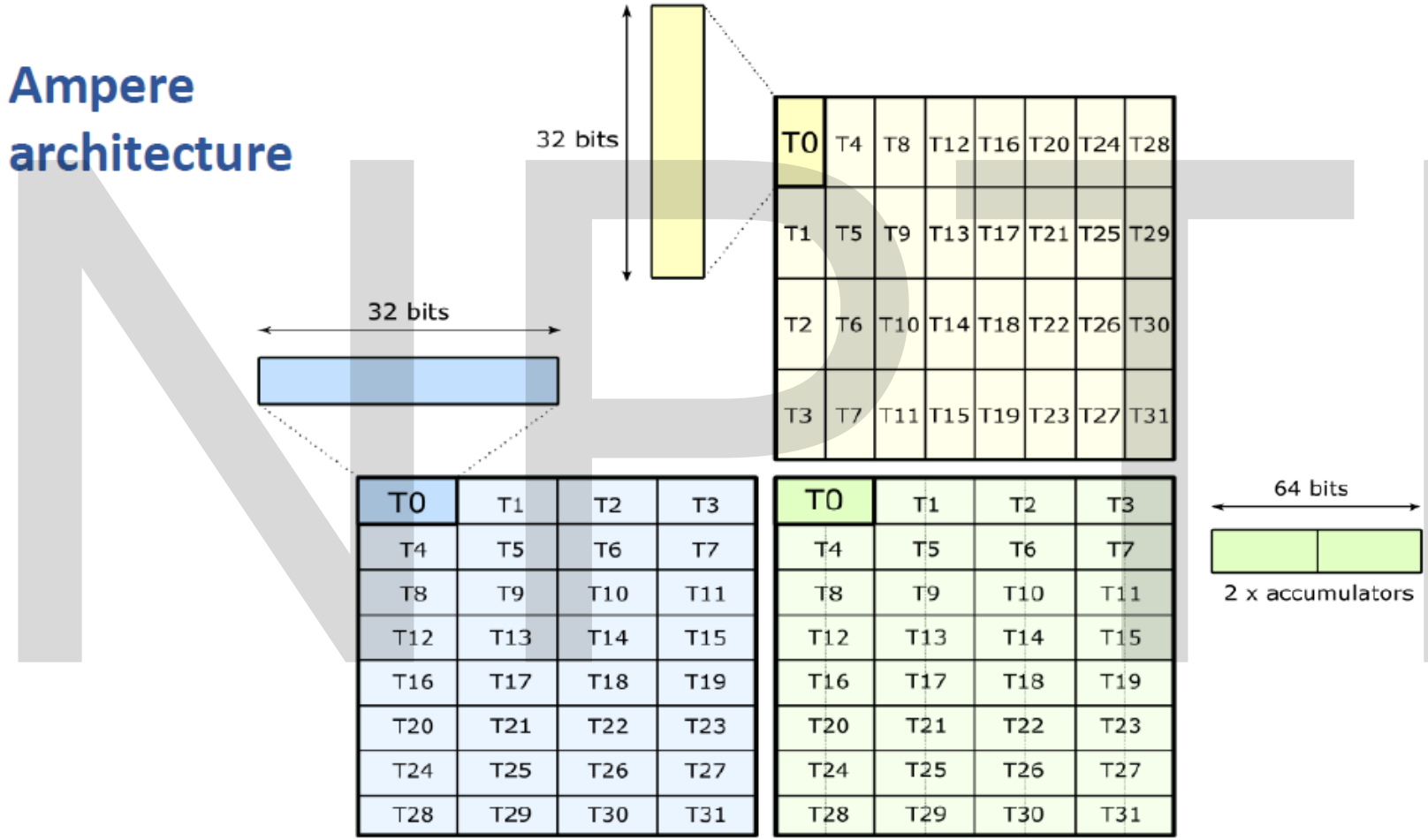
NPTEL

Programming Hierarchy in GPUs



NPTEL

Work Distribution among Warp Threads



- wmma instructions take parameters for shape and data types
- shape: m8n4k4
- Data type: f32 f32f64
- Only a limited number of shapes and data types are allowed by the ISA

Agenda

- Recap
- **Training Large Models with Limited GPU Memory**

NPTEL

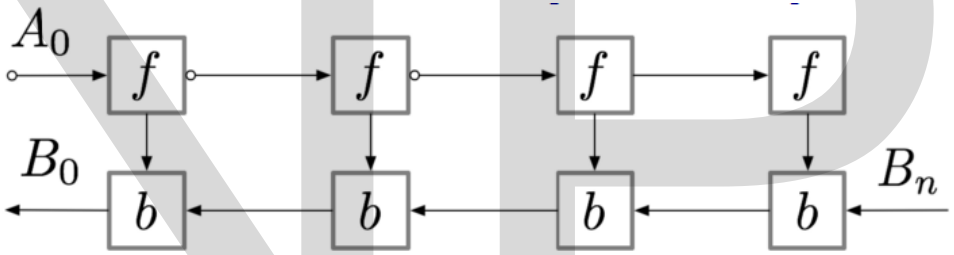
Large Model training limited by GPU memory capacity

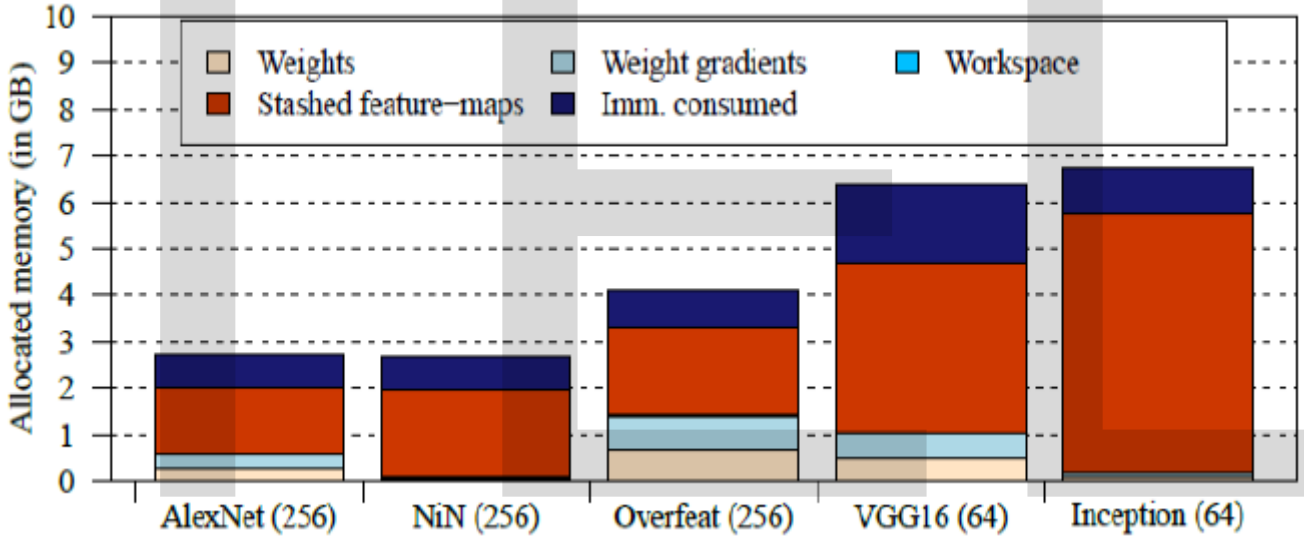
- Since 2015, TFLOPs in NVIDIA GPU has grown 32x
- Memory capacity has grown 2.5x (max out at 80GB today)
- DNN training jobs are often limited by the GPU memory capacity
- 8.8% of DL jobs failure is due to GPU OOM in Microsoft cloud
- Most common reason of all DL job failure

NPTEL

Gradient Checkpointing to reduce Memory Footprint

- **Observation: Significant memory overheads for keeping o/p of forward pass (fmap) for gradient computation during backward pass [1]**





[1] Jain, Animesh, et al. "Gist: Efficient data encoding for deep neural network training." *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018.

Gradient Checkpointing to reduce Memory Footprint

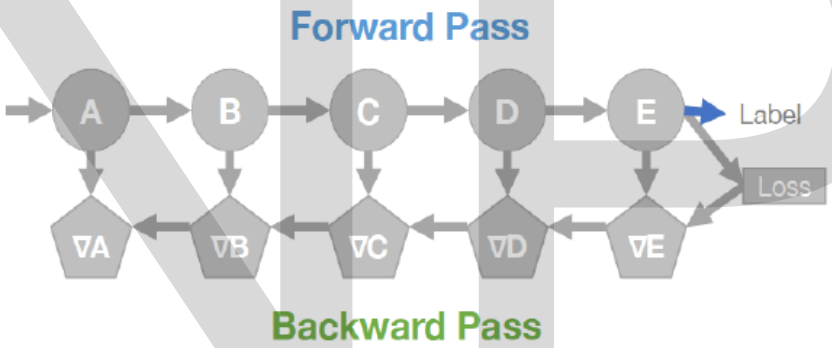
- Idea: Don't keep output tensors (fmaps) for the backpropagation
  - During backpropagation re-compute output of forward pass
  - Lowers memory footprint but increases computation cost

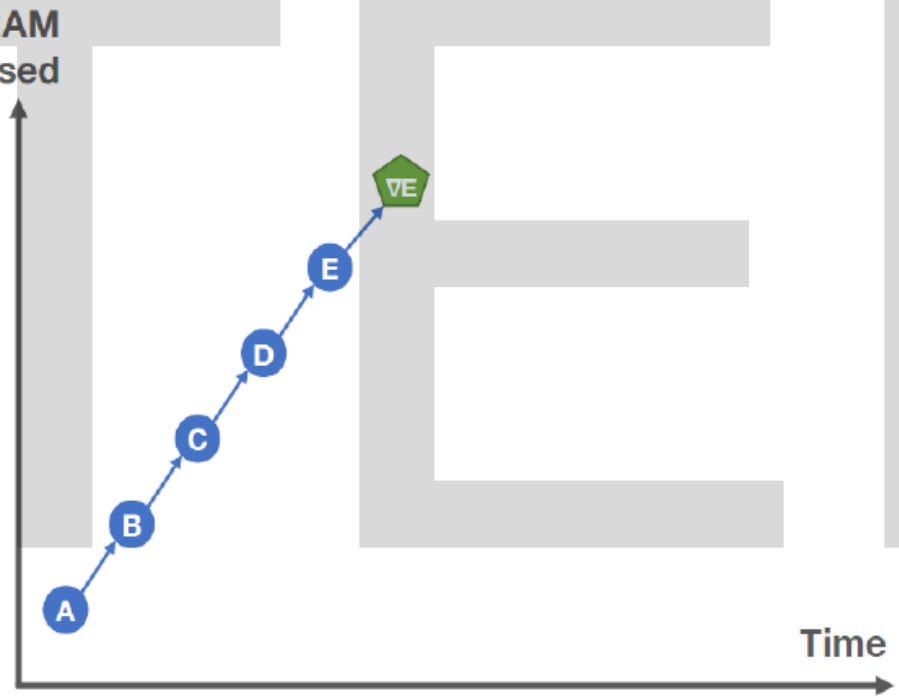
- Which fmaps to keep (checkpoint) and which ones to free + recompute during backward pass?

NPTEL

Baseline: Highest Memory Footprint

- If all layers are kept in RAM, memory footprint is the highest; compute is lowest

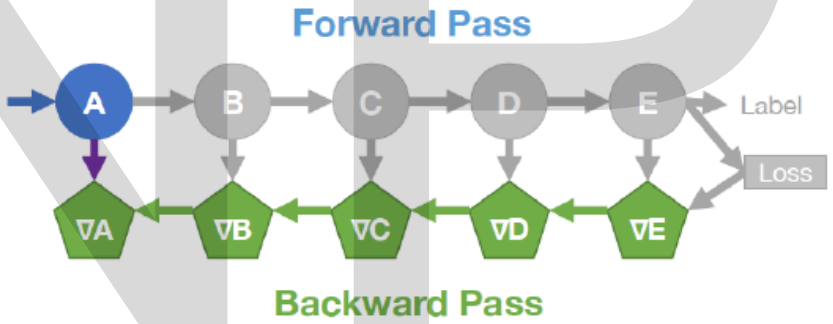


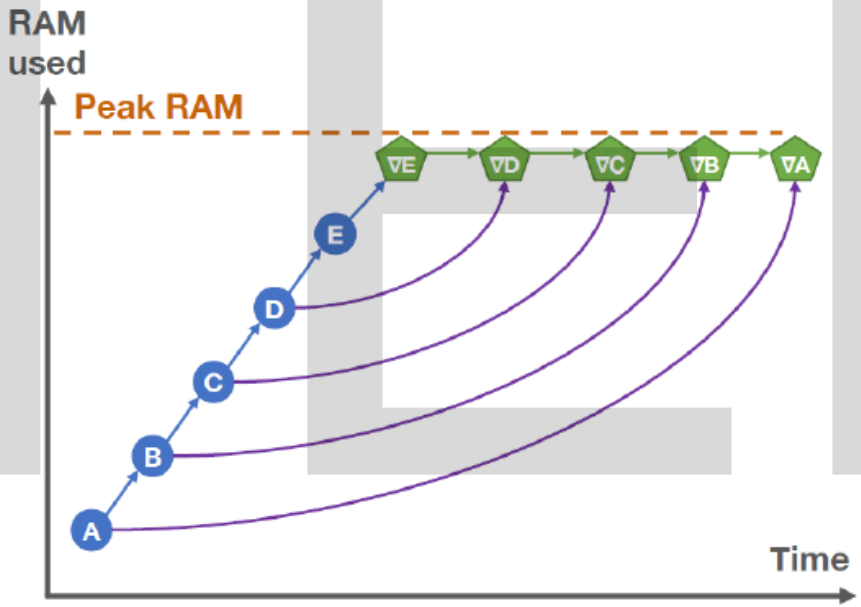


NPTEL

Baseline: Highest Memory Footprint

- If all layers are kept in RAM, memory footprint is the highest; compute is lowest

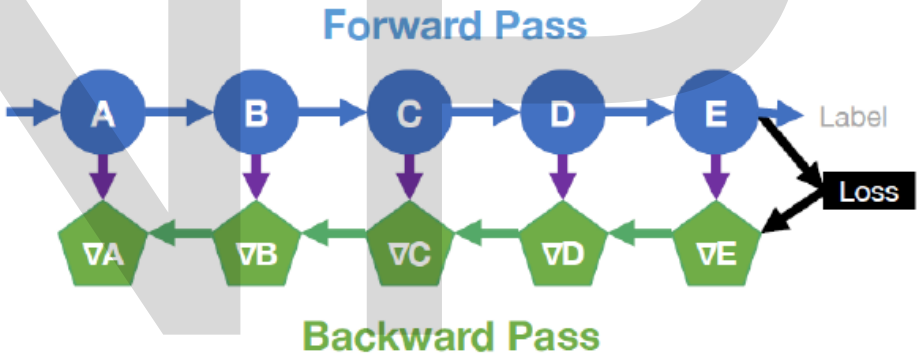


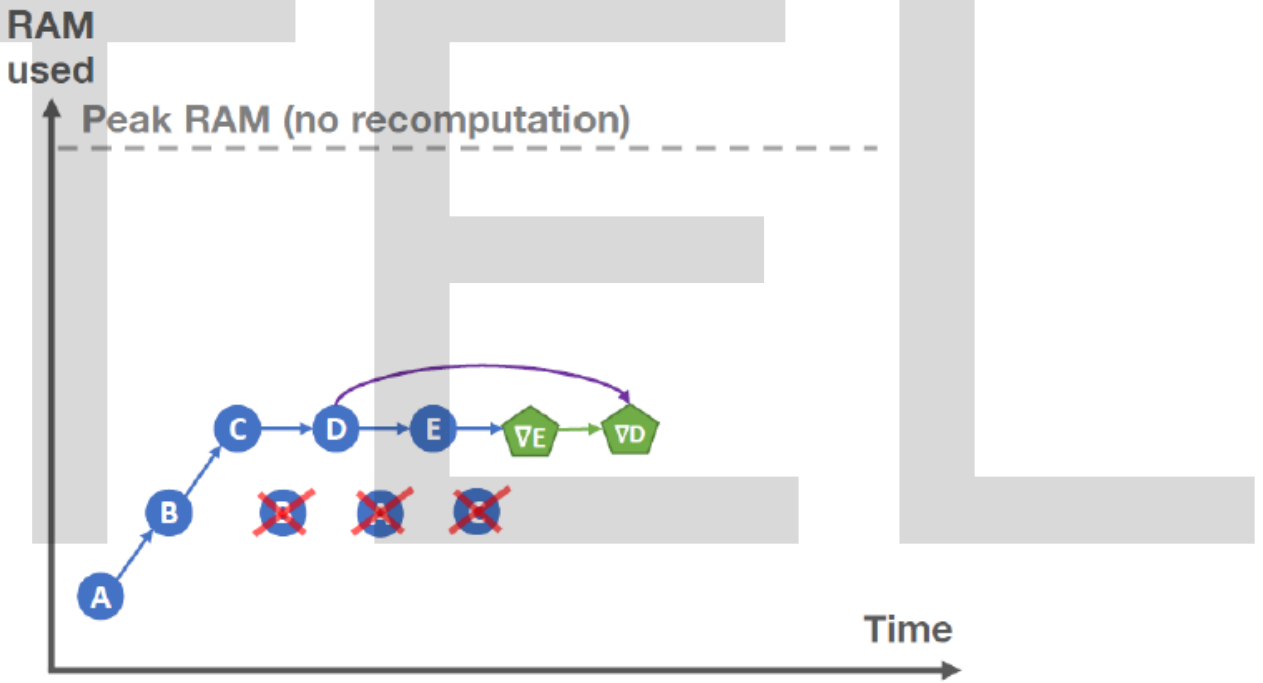


NPTEL

De-allocate tensors ASAP

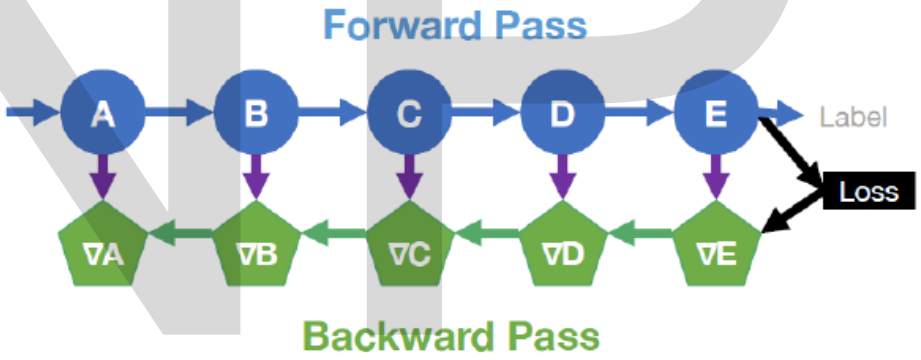
- Recompute all layers
- Memory footprint is the lowest

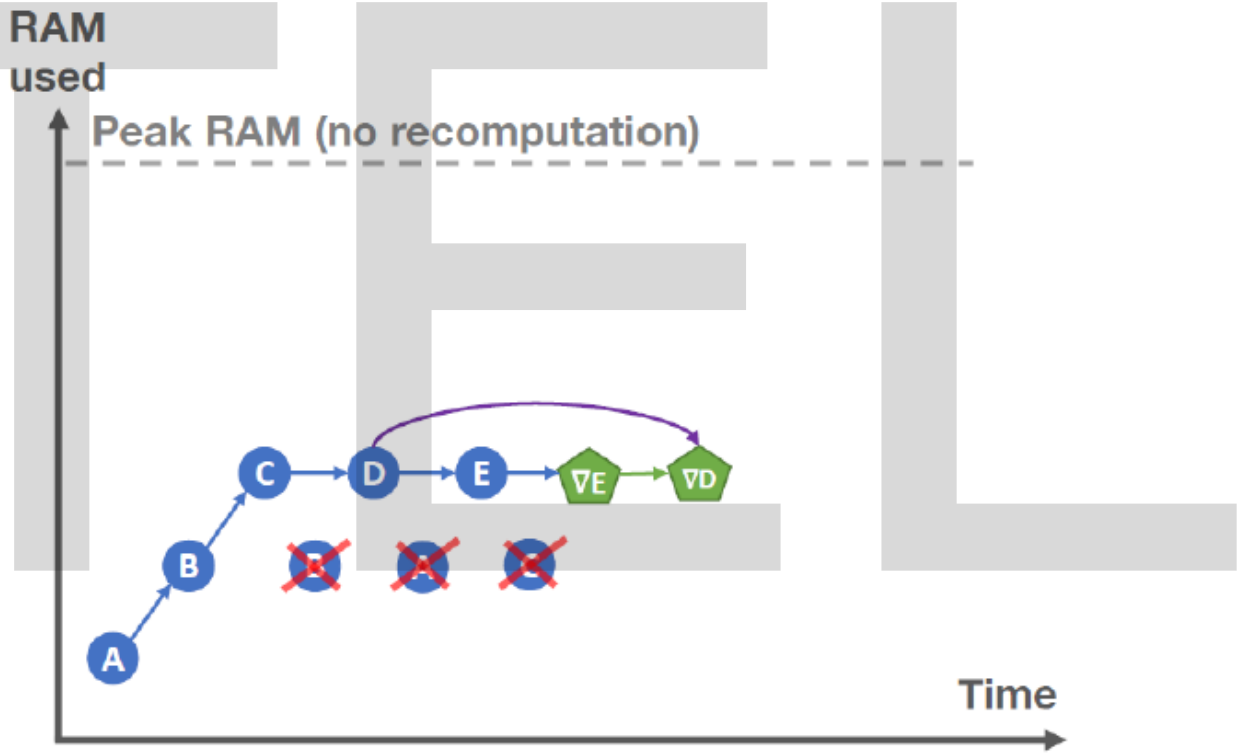




De-allocate tensors ASAP

- Recompute all layers
- Memory footprint is the lowest





Which fmaps to keep (checkpoint)?

- Things to consider
  - Different layers can have different compute cost
  - Different layers can have different memory footprint
  - Wide variety of network architecture with skipconnections
    - Determines earliest one can de-allocate the tensor
    - Determines where all the fmapmay be needed in the backward pass

NPTEL

Solution: Optimization across Layers [2]

- **Use profiling for memory and compute cost of each layer**
- **Use Mixed Integer Linear Programming to find what to checkpoint**

Minimize forward + backward cost

$$\min_{S,R,U} \sum \sum C_i \, R_{t,i}$$

Correctness

$$R_{t,j} \leq R_{t,i} + S_{t,i}$$

$$S_{t,i} \leq R_{t-1,i} + S_{t-1,i}$$

Memory limit

$$U_{t,k} \leq \text{budget}, \ldots$$

Constrain memory via an implicit variable to model memory usage at each stage

Decision variables

$S_{t,i} \in \{0,1\}$	Layer i stored for stage t
$R_{t,i} \in \{0,1\}$	Layer i (re)computed in stage t
$U_{t,i} \in \mathbb{R}_+$	Memory usage in stage t

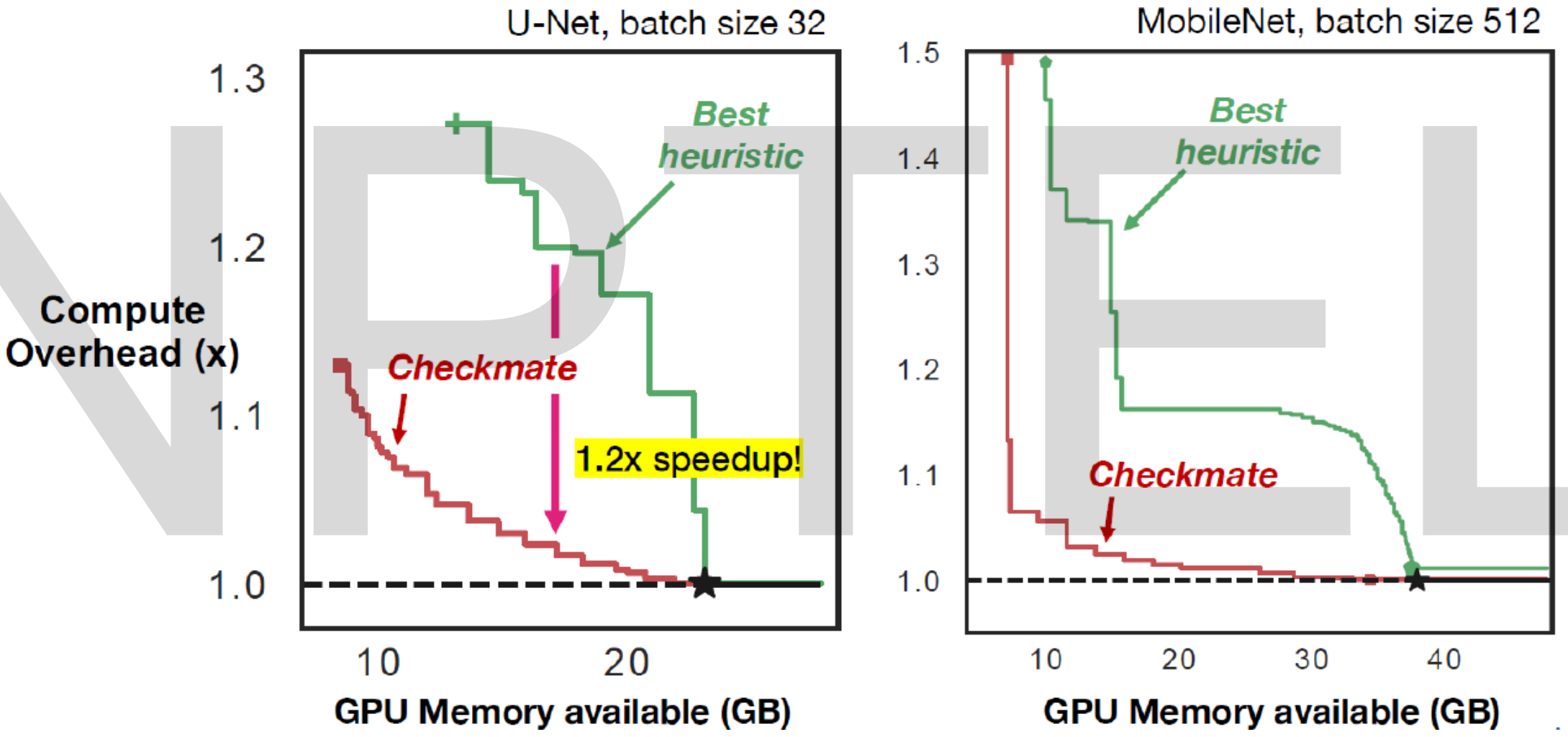


checkmateai.github.io

NPTEL

[1] Jain, Paras, et al. "Checkmate: Breaking the memory wall with optimal tensor rematerialization." *Proceedings of Machine Learning and Systems* 2 (2020): 497-511.

Results of Optimization



NPTEL

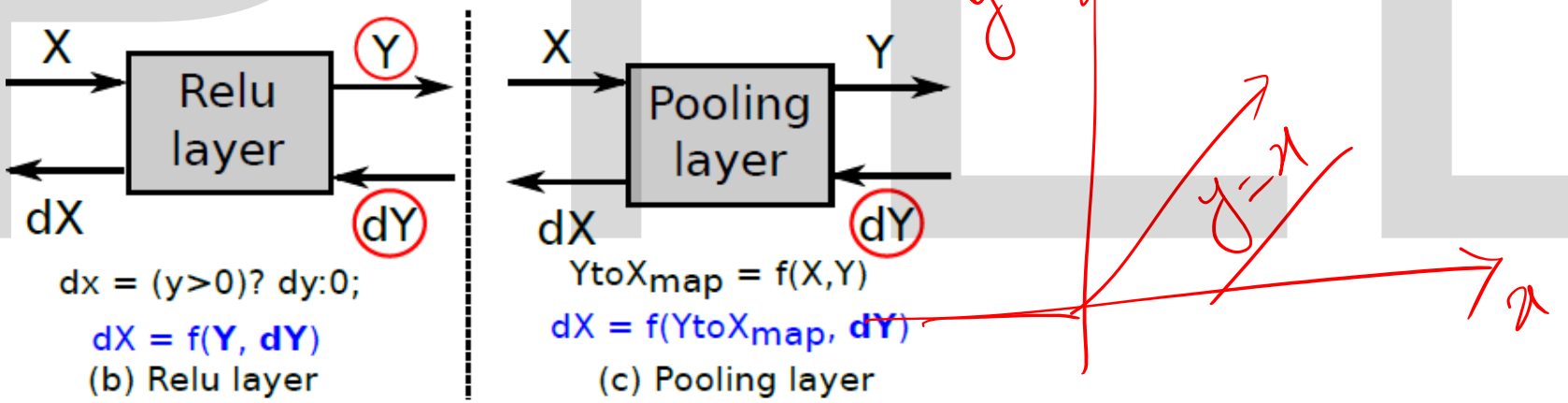
Operator Optimizations for Reducing Memory Footprint

- Each operator (e.g., Conv) needs its own memory during compute
  - Input, workspace, weights, output
- Choose different implementation of an operator to trade-off its memory needs and compute
  - For example, Convolution can have different implementations - Winograd, FFT, GEMM
- Compress a fmap and decompress during backward pass
  - ReLU introduces sparsity. Lossless encoding of fmap with high sparsity.
  - Decode during backward pass (rather than re-compute)

NPTEL

Leverage Model-specific Encoding [3]

- **Observations:**
  - **In backward pass for ReLU, dY is passed to dX, if Y is positive**
  - **If MaxPool follows ReLU, dX can be computed without ReLU output**
- **Optimization: ReLU output encoded in 1 bit instead of 32 bits**



[3] Jain, Animesh, et al. "Gist: Efficient data encoding for deep neural network training." *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018.

Gradient Accumulation

- Break a batch into “n” mini-batches
- The size of fmaps reduces by n
- Run the forward and backward pass for each mini-batch
- But don’t update the weights
- Accumulate(add) gradients at each step
- Update the weights after the nth step (i.e., after batch is complete)



NPTEL

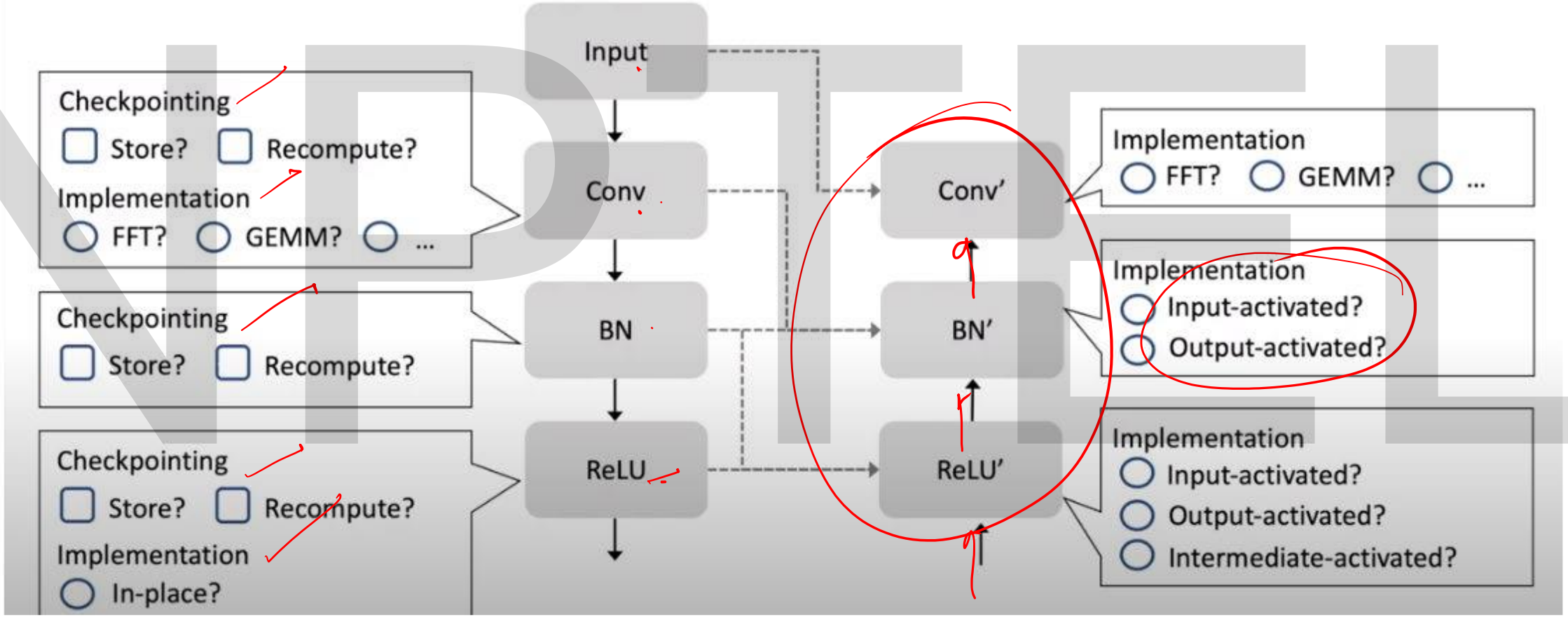
Which Optimization should be Applied Where? [4]

- Automatic framework to minimize the memory without large compute overhead of deep networks
- Combines multiple previously known technique
  - Checkpointing + recompute [e.g., Checkmate]
  - Choosing memory efficient operator [e.g., Gist]
  - Takes into consideration computation overhead
  - Fits into integer linear program
- Can help minimize execution time under a given memory budget

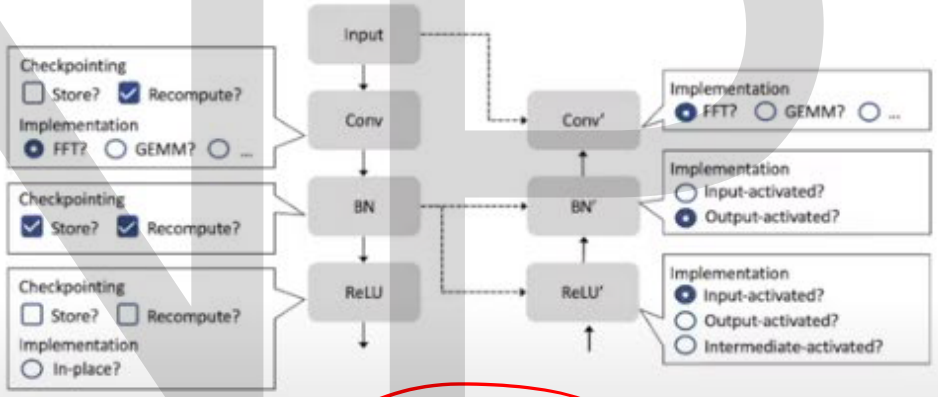
NPTEL

[4] Shah, Aashaka, et al. "Memory Optimization for Deep Networks." *International Conference on Learning Representations*.

Memory Optimization Framework



Memory Optimization Framework



MONeT formulation

Boolean linear programming problem

Objective: Minimize computation time

Memory constraints

Peak forward memory ≤ Memory budget

Peak backward memory ≤ Memory budget

Peak recomputation memory ≤ Memory budget

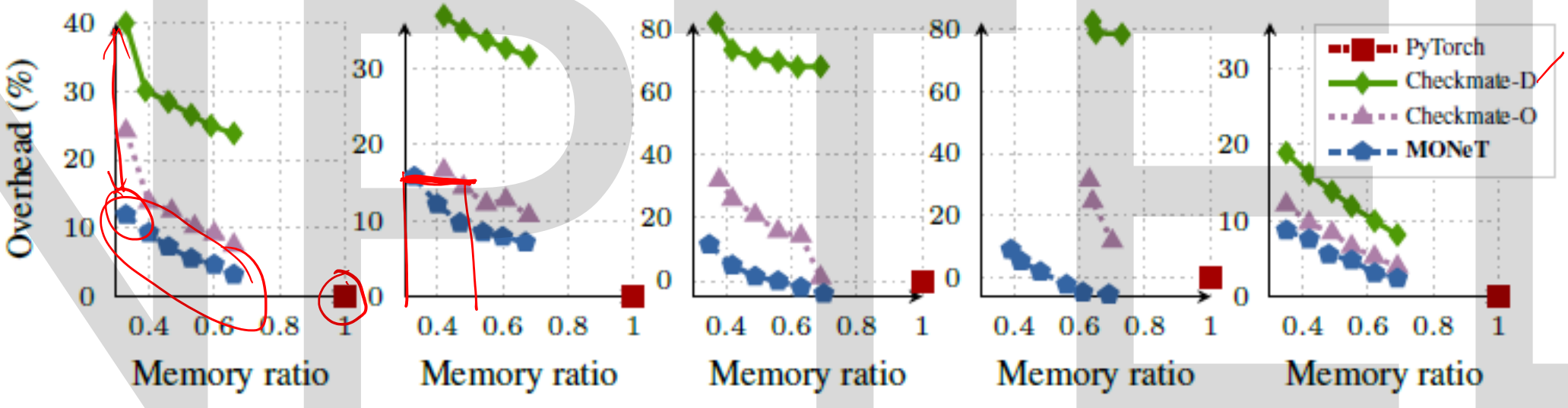
Checkpointing constraints

Only use operator output if present in memory

Execution schedule

Off-the-shelf solver

Results of Memory Optimization Framework



(a) **ResNet-50** (184) (b) **GoogleNet** (320) (c) **UNet** (11) (d) **VGG-16** (176) (e) **Mobile-V2** (272)

Further Read

- Key parameters for a DL training job are user provided
  - Batch size
  - Learning rate
- Assumption: Cannot change the user provided DL job configuration
- But, end goal is to train with good accuracy, fast training, low resource usage (memory)
- Deciding the batch size as a function of hardware parameters [5]

NPTEL

[5] Qiao, Aurick, et al. "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning." *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*. 2021.



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)